

Hardware Efficient Ansätze for QML

QML

Contents

1	Overview	4
2	What Makes an Ansatz “Hardware Efficient”?	4
3	Native Gate Sets	4
3.1	IBM Native Gates	4
3.2	Google Native Gates	5
3.3	Trapped Ion Native Gates	5
4	Gate Decomposition	5
4.1	Single-Qubit Gate Decomposition	6
4.2	Two-Qubit Gate Decomposition	6
4.3	SWAP Decomposition	6
5	Transpilation	7
5.1	The Transpilation Pipeline	7
5.2	Transpilation Stages in Detail	7
5.3	Optimization Levels in Qiskit	7
5.4	Impact on Circuit Depth: Before vs After	8
6	SWAP Networks for Connectivity Constraints	9
6.1	SWAP Cost by Topology	9
6.2	Odd-Even SWAP Network	9
6.3	Linear SWAP Strategy	9
7	RealAmplitudes Ansatz	9
7.1	Structure	10
7.2	Circuit Diagram	10
7.3	Parameter Count	10
7.4	Gate Count	11
7.5	When to Use RealAmplitudes	11
8	EfficientSU2 Ansatz	11

8.1	Structure	11
8.2	Circuit Diagram	11
8.3	Parameter Count	12
8.4	Comparison with RealAmplitudes	12
9	TwoLocal Ansatz	12
9.1	Configuration	12
10	Hardware-Efficient vs Problem-Inspired Ansätze	13
10.1	The Tradeoff	13
10.2	When to Use Which	14
11	Reducing Two-Qubit Gate Count	14
11.1	Gate Cancellation	14
11.2	Template Matching	14
11.3	Commutation Analysis	15
11.4	Two-Qubit Block Consolidation	15
12	QAOA as a Hardware-Aware Ansatz	15
12.1	QAOA Structure	15
12.2	Why QAOA Can Be Hardware-Efficient	16
13	Practical Ansatz Design Checklist	16
13.1	Step 1: Know Ur Hardware	16
13.2	Step 2: Compute Ur Noise Budget	17
13.3	Step 3: Choose Ansatz Family	17
13.4	Step 4: Configure Entanglement Pattern	17
13.5	Step 5: Set Number of Layers	17
13.6	Step 6: Transpile and Verify	18
14	The Qiskit Pass Manager	18
14.1	Transpilation Passes	18
14.2	Optimization Passes in Detail	18
15	Circuit Depth Optimization Techniques	19
15.1	Parallelization	19
15.2	Entanglement Reduction	19
15.3	Layer-Wise Training	20
15.4	Parameter Sharing	20

16	Impact of Transpilation: Detailed Example	20
16.1	The Abstract Circuit	20
16.2	Transpilation to Linear Chain	20
16.3	Before/After Comparison	21
17	Advanced: Noise-Aware Ansatz Design	21
17.1	Qubit Quality Variation	21
17.2	Error-Aware Entanglement	22
17.3	Dynamical Decoupling	22
18	Summary and Design Principles	22

1 Overview

last lecture we learned that real quantum hardware is noisy, has limited connectivity, and can only faithfully execute circuits up to a certain depth. this lecture is about wt u do about it: how to design variational ansatze that respect hardware constraints while still being expressive enough for QML.

the central tension is simple: u want ur ansatz to be expressive (so it can represent complex functions) but shallow (so noise doesnt destroy the signal). hardware-efficient ansatze try to get the maximum expressiveness per unit of circuit depth by using only gates that the hardware natively supports and only connections that exist in the physical qubit topology.

well cover the main hardware-efficient ansatz families (RealAmplitudes, EfficientSU2, TwoLocal), understand transpilation and how it affects circuit depth, and develop practical guidelines for choosing and designing ansatze for specific hardware.

the punchline: the “best” ansatz isnt the most expressive one — its the one that gives the most useful expressiveness within ur noise budget.

2 What Makes an Ansatz “Hardware Efficient”?

Definition 2.1 (Hardware-Efficient Ansatz). an ansatz $U(\theta)$ is hardware-efficient if:

- (1) it uses only gates from the hardware’s native gate set
- (2) it respects the hardware’s qubit connectivity (entangling gates only between physically connected qubits)
- (3) it minimizes circuit depth for a given number of parameters

why does this matter? bcs any gate thats not in the native gate set must be decomposed into native gates, adding depth. any entangling gate between non-connected qubits requires SWAP routing, adding even more depth. and every additional gate adds noise.

Example 2.1. suppose ur ideal ansatz uses a CNOT between qubits 0 and 5 on a heavy-hex lattice where the shortest path from q_0 to q_5 is 4 edges. u need 3 SWAP gates to bring the qubits adjacent, each SWAP costs 3 CNOTs, so that single logical CNOT becomes $3 \times 3 + 1 = 10$ physical CNOTs. if ur 2Q gate error is 0.5%, that one “gate” now has error $\approx 1 - (1 - 0.005)^{10} \approx 4.9\%$. thats terrible.

Key Point. a hardware-efficient ansatz avoids this blowup by construction. it only uses gates and connections that map directly to the hardware, so wt u design is (approximately) wt gets executed.

3 Native Gate Sets

every quantum processor has a small set of gates it can physically implement. everything else gets decomposed into these native gates during transpilation.

3.1 IBM Native Gates

IBM’s superconducting processors use the following native gate set:

Gate	Matrix	Type
$R_z(\lambda)$	$\begin{pmatrix} e^{-i\lambda/2} & 0 \\ 0 & e^{i\lambda/2} \end{pmatrix}$	1Q, virtual (zero error)
$\sqrt{X}$ (SX)	$\frac{1}{2} \begin{pmatrix} 1+i & 1-i \\ 1-i & 1+i \end{pmatrix}$	1Q, physical
X	$\begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}$	1Q, physical
ECR	$\frac{1}{\sqrt{2}} \begin{pmatrix} 0 & 0 & 1 & i \\ 0 & 0 & i & 1 \\ 1 & -i & 0 & 0 \\ -i & 1 & 0 & 0 \end{pmatrix}$	2Q, physical

Key Point. R_z gates on IBM hardware r “virtual” — they r implemented by adjusting the phase of subsequent pulses rather than applying a physical pulse. this means R_z gates have essentially zero error and zero duration. design ur ansatz to use lots of R_z and minimize physical gates (SX, ECR).

3.2 Google Native Gates

Google’s Sycamore processor uses:

- single-qubit: arbitrary $R_z(\theta)$, $\sqrt{W} = \frac{R_x(\pi/2)+R_y(\pi/2)}{\sqrt{2}}$
- two-qubit: $\sqrt{i}\text{SWAP}$ and SYC (Sycamore gate)

the Sycamore gate is:

$$\text{SYC} = \text{fSim}\left(\frac{\pi}{2}, \frac{\pi}{6}\right) = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 0 & -i & 0 \\ 0 & -i & 0 & 0 \\ 0 & 0 & 0 & e^{-i\pi/6} \end{pmatrix}$$

3.3 Trapped Ion Native Gates

IonQ and Quantinuum use:

- single-qubit: $R(\theta, \phi) = \exp(-i\frac{\theta}{2}(\cos\phi\sigma_x + \sin\phi\sigma_y))$ (arbitrary axis in XY plane)
- two-qubit: MS gate $\text{MS}(\theta) = \exp(-i\frac{\theta}{4}\sigma_x \otimes \sigma_x)$ (IonQ) or ZZ interaction (Quantinuum)

4 Gate Decomposition

when ur circuit uses gates that arent in the native set, the transpiler must decompose them. understanding these decompositions helps u estimate the true cost of ur ansatz.

4.1 Single-Qubit Gate Decomposition

any single-qubit unitary can be decomposed using the Euler decomposition:

$$U = e^{i\alpha} R_z(\beta) R_y(\gamma) R_z(\delta)$$

on IBM hardware, since $R_y(\gamma) = R_z(-\pi/2) \cdot SX \cdot R_z(\pi/2 - \gamma) \cdot SX \cdot R_z(\gamma - \pi)$ (approximately), any single-qubit gate costs at most 2 SX gates and 3 R_z gates. since R_z is virtual, the physical cost is just 2 SX gates.

Example 4.1. decomposing a Hadamard gate on IBM hardware:

$$H = R_z(\pi) \cdot SX \cdot R_z(\pi)$$

cost: 1 SX gate + 2 virtual R_z gates = effectively 1 physical gate.

4.2 Two-Qubit Gate Decomposition

this is where it gets expensive. the standard decomposition of CNOT into ECR:

$$\text{CNOT}_{01} = (I \otimes R_z(-\pi/2)) \cdot \text{ECR}_{01} \cdot (R_x(\pi) \otimes I)$$

so a CNOT costs exactly 1 ECR gate plus single-qubit corrections (which r basically free).

a general two-qubit unitary $U \in SU(4)$ can require up to 3 CNOT gates (or 3 ECR gates) via the KAK decomposition:

$$U = (A_1 \otimes B_1) \cdot \text{CNOT} \cdot (A_2 \otimes B_2) \cdot \text{CNOT} \cdot (A_3 \otimes B_3) \cdot \text{CNOT} \cdot (A_4 \otimes B_4)$$

Theorem 4.1 (Cartan/KAK Decomposition). any $U \in SU(4)$ can be written as:

$$U = (k_1 \otimes k_2) \cdot \exp(i(a\sigma_x \otimes \sigma_x + b\sigma_y \otimes \sigma_y + c\sigma_z \otimes \sigma_z)) \cdot (k_3 \otimes k_4)$$

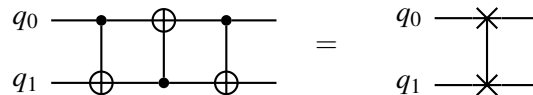
where $k_i \in SU(2)$ and $a, b, c \in \mathbb{R}$. the parameters (a, b, c) determine how many CNOT gates r needed: 0 if $(a, b, c) = (0, 0, 0)$, 1 if $b = c = 0$, 2 if $c = 0$, and 3 in general.

4.3 SWAP Decomposition

a SWAP gate decomposes into 3 CNOT gates:

$$\text{SWAP} = \text{CNOT}_{01} \cdot \text{CNOT}_{10} \cdot \text{CNOT}_{01}$$

this is the circuit:

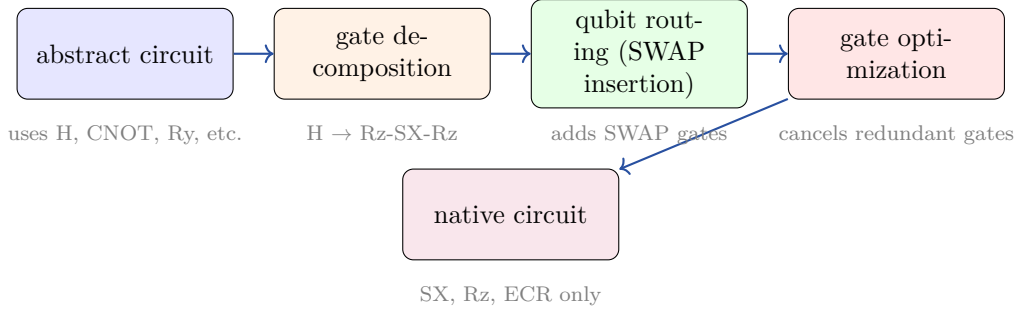


at $\sim 0.5\%$ error per CNOT, a SWAP costs $\sim 1.5\%$ error. thats why minimizing SWAPs is so critical.

5 Transpilation

transpilation is the process of converting ur abstract circuit into a hardware-executable circuit. its one of the most important steps in any QML pipeline, and understanding it is essential for getting good results.

5.1 The Transpilation Pipeline



5.2 Transpilation Stages in Detail

stage 1: unrolling and decomposition. every gate in ur circuit is decomposed into the native gate set. a hadamard becomes $R_z\text{-}SX\text{-}R_z$, a $R_y(\theta)$ becomes $R_z\text{-}SX\text{-}R_z$ combinations, a CNOT becomes an ECR with corrections.

stage 2: routing. the transpiler maps logical qubits to physical qubits and inserts SWAP gates wherever the circuit requires a two-qubit gate between non-adjacent qubits. this is the stage that can massively increase circuit depth. the routing problem is NP-hard in general, but good heuristics (like SABRE — SWAP-based bidirectional heuristic search) work well in practice.

stage 3: optimization. after routing, the circuit often has redundant gates that can be canceled or simplified:

- adjacent inverse gates cancel: $R_z(\theta)R_z(-\theta) = I$
- adjacent same-axis rotations merge: $R_z(\alpha)R_z(\beta) = R_z(\alpha + \beta)$
- template matching: known circuit identities r applied to simplify subcircuits
- two-qubit gate cancellation: back-to-back CNOTs cancel

5.3 Optimization Levels in Qiskit

Qiskit's `transpile` function supports four optimization levels:

Level	What It Does	Speed
0	minimal: just decompose to native gates and do basic routing. no optimization.	fastest
1	light optimization: merge adjacent rotations, cancel inverse gates, basic layout selection. this is the default.	fast
2	medium: adds more aggressive gate cancellation, template matching, commutation analysis.	moderate
3	heavy: tries multiple initial layouts, uses the best one. most aggressive optimization including two-qubit block consolidation and resynthesis.	slowest

Key Point. for QML, always use optimization level 3 for production runs. the extra compilation time (seconds to minutes) is negligible compared to the execution time, and it can reduce ur two-qubit gate count by 20–50%. for rapid prototyping during development, level 1 is fine.

5.4 Impact on Circuit Depth: Before vs After

lets see how transpilation affects a simple QML circuit.

Example 5.1 (Transpilation Impact). consider a 4-qubit, 2-layer ansatz with all-to-all CNOT connectivity on a linear chain topology ($q_0 - q_1 - q_2 - q_3$):

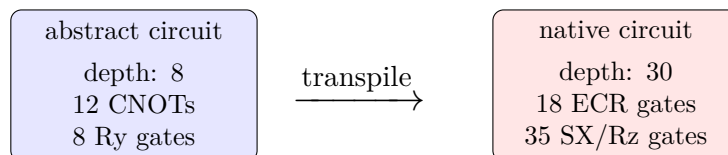
before transpilation:

- 4 qubits, 2 layers
- each layer: 4 R_y gates + 6 CNOT gates (all pairs)
- total: 8 R_y + 12 CNOT = 20 abstract gates
- abstract depth: ~ 8

after transpilation (level 3, linear chain):

- non-adjacent CNOTs (q_0-q_2 , q_0-q_3 , q_1-q_3) need SWAP routing
- 6 CNOTs become $\sim 12-18$ native ECR gates (due to SWAPs)
- single-qubit gates: $\sim 30-40$ (SX + R_z)
- native depth: $\sim 25-35$

the depth increased by $\sim 3-4x$ due to SWAP routing. this is why hardware-efficient ansatze that only use nearest-neighbor connections r so important.



depth increased 3.75x due to SWAP routing

6 SWAP Networks for Connectivity Constraints

when ur circuit requires entangling gates between non-adjacent qubits, the transpiler inserts SWAP gates to “move” qubit states to adjacent positions. understanding the SWAP cost for different topologies helps u design better ansatze.

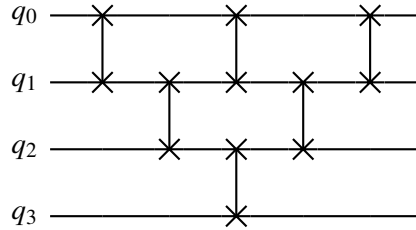
6.1 SWAP Cost by Topology

for a circuit that needs all-to-all connectivity on n qubits:

Topology	Max SWAPs per 2Q Gate	Overhead for All-to-All
all-to-all (ions)	0	none
fully connected	0	none
square grid ($\sqrt{n} \times \sqrt{n}$)	$O(\sqrt{n})$	$O(n^{3/2})$
heavy-hex	$O(\sqrt{n})$	$O(n^{3/2})$
linear chain	$O(n)$	$O(n^2)$

6.2 Odd-Even SWAP Network

a common pattern for implementing all-to-all connectivity on a linear chain is the odd-even transposition sort. after n rounds of alternating odd and even pair SWAPs, any permutation can be achieved:



after these SWAP layers, all qubit states have been permuted such that non-adjacent pairs become adjacent at some point during the network. u can interleave ur entangling gates with the SWAPs.

6.3 Linear SWAP Strategy

an alternative is the “linear reversal” strategy: repeatedly SWAP adjacent pairs to shift qubit states along the chain. to bring q_0 next to q_3 on a 4-qubit linear chain, u SWAP q_0 through q_1 , then through q_2 :

depth cost: $n - 1$ SWAP layers to bring the farthest qubits together. each SWAP layer adds depth 3 (in CNOT) or depth 1 (in native SWAP gates if available).

7 RealAmplitudes Ansatz

RealAmplitudes is one of the simplest and most commonly used hardware-efficient ansatze in Qiskit. its a good starting point for most QML tasks.

7.1 Structure

Definition 7.1 (RealAmplitudes Ansatz). the RealAmplitudes ansatz consists of alternating layers of:

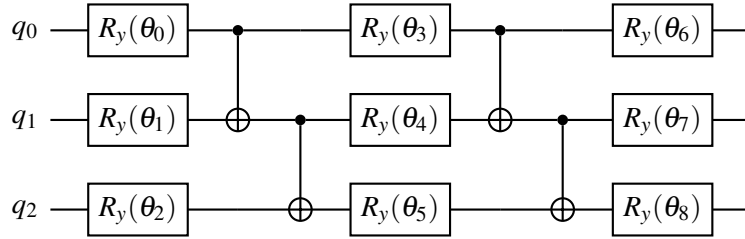
- (1) $R_y(\theta)$ rotations on each qubit
- (2) CNOT entangling gates between pairs of qubits (configurable: linear, full, circular, etc.)

the name comes from the fact that R_y rotations and CNOTs preserve real amplitudes — if u start from $|0\rangle^{\otimes n}$ (which has real amplitudes), the output state always has real amplitudes. the state can be written as:

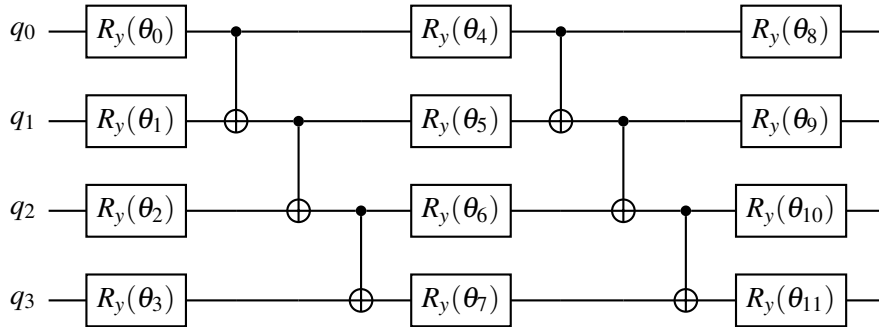
$$|\psi(\theta)\rangle = \prod_{l=1}^L \left[\text{Ent}_l \cdot \bigotimes_{i=0}^{n-1} R_y(\theta_i^{(l)}) \right] \cdot \bigotimes_{i=0}^{n-1} R_y(\theta_i^{(0)}) |0\rangle^{\otimes n}$$

7.2 Circuit Diagram

3-qubit RealAmplitudes with 2 layers and linear entanglement:



4-qubit RealAmplitudes with 2 layers and linear entanglement:



7.3 Parameter Count

for n qubits and L entangling layers:

$$N_{\text{params}} = n(L+1)$$

(each of the $L+1$ rotation layers has n parameters).

for the 4-qubit, 2-layer example above: $4 \times 3 = 12$ parameters.

7.4 Gate Count

- single-qubit gates: $n(L+1)$ R_y gates
- two-qubit gates: $L(n-1)$ CNOTs (for linear entanglement)
- total depth: $\sim 3L+1$ (each entangling layer has depth $\sim 2-3$)

7.5 When to Use RealAmplitudes

- ur problem has real-valued wave functions (many optimization problems)
- u want the simplest possible ansatz as a baseline
- ur hardware budget is very tight and u need minimal gate count
- ur training data is real-valued and u dont expect complex phases to help

Intuition. RealAmplitudes is the “linear regression” of quantum ansatze — simple, interpretable, and a good first thing to try. if it doesnt work, move to smtg more expressive. if it does work, u probably dont need smtg more complex.

8 EfficientSU2 Ansatz

EfficientSU2 is a more expressive hardware-efficient ansatz that can produce arbitrary single-qubit states (not just real amplitudes).

8.1 Structure

Definition 8.1 (EfficientSU2 Ansatz). the EfficientSU2 ansatz consists of alternating layers of:

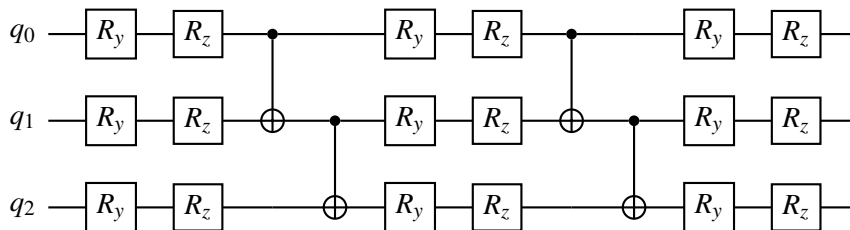
- (1) two single-qubit rotations per qubit (by default $R_y(\theta)$ followed by $R_z(\phi)$) — this gives full SU(2) coverage on each qubit
- (2) CNOT entangling gates (configurable connectivity)

the state is:

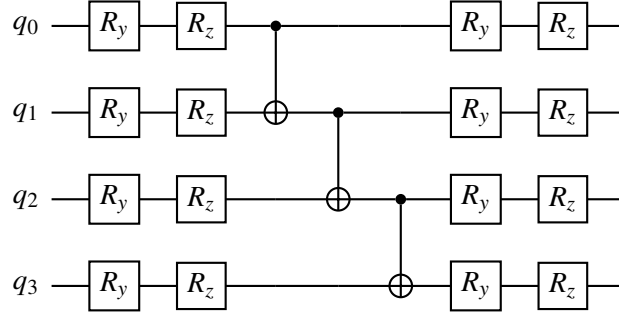
$$|\psi(\theta, \phi)\rangle = \prod_{l=1}^L \left[\text{Ent}_l \cdot \bigotimes_{i=0}^{n-1} R_z(\phi_i^{(l)}) R_y(\theta_i^{(l)}) \right] \cdot \bigotimes_{i=0}^{n-1} R_z(\phi_i^{(0)}) R_y(\theta_i^{(0)}) |0\rangle^{\otimes n}$$

8.2 Circuit Diagram

3-qubit EfficientSU2 with 2 layers and linear entanglement:



4-qubit EfficientSU2 with 1 layer and linear entanglement:



8.3 Parameter Count

for n qubits and L entangling layers:

$$N_{\text{params}} = 2n(L + 1)$$

(each rotation layer has $2n$ parameters — one R_y and one R_z per qubit).

for 4 qubits, 2 layers: $2 \times 4 \times 3 = 24$ parameters (double that of RealAmplitudes).

8.4 Comparison with RealAmplitudes

Property	RealAmplitudes	EfficientSU2
rotations per qubit per layer	1 (R_y)	2 ($R_y + R_z$)
parameters (n qubits, L layers)	$n(L + 1)$	$2n(L + 1)$
can produce complex amplitudes	no	yes
single-qubit gate cost	lower	higher ($\sim 2x$)
expressibility	lower	higher
training difficulty	easier	harder (more parameters)
typical use	real-valued problems	general QML

Key Point. EfficientSU2 has twice the parameters of RealAmplitudes for the same depth and qubit count. this means more expressiveness but also harder training (more parameters to optimize). in practice, start with RealAmplitudes. if the model isnt expressive enough, switch to EfficientSU2 before increasing the number of layers.

9 TwoLocal Ansatz

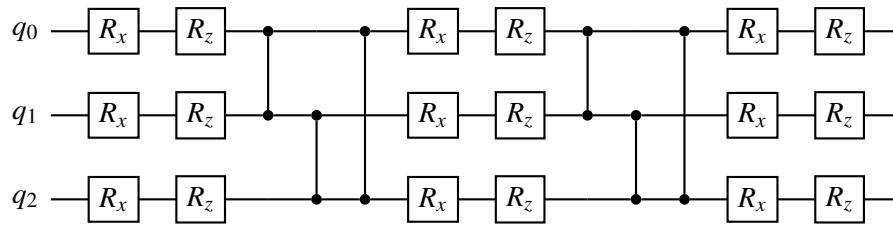
TwoLocal is the general framework that both RealAmplitudes and EfficientSU2 r special cases of. it gives u full control over the ansatz structure.

9.1 Configuration

Definition 9.1 (TwoLocal Ansatz). a TwoLocal ansatz is specified by:

- (1) rotation blocks: which single-qubit gates to use (e.g., $[R_y]$, $[R_y, R_z]$, $[R_x, R_y, R_z]$)
- (2) entanglement blocks: which two-qubit gates to use (e.g., CNOT, CZ, R_{zz})
- (3) entanglement pattern: which qubit pairs to entangle (linear, full, circular, pairwise, custom list)
- (4) number of repetitions: how many layers L
- (5) skip final rotation: whether to include the last rotation layer

Example 9.1 (Custom TwoLocal). a TwoLocal with rotation blocks $[R_x, R_z]$, entanglement block CZ, circular entanglement, 2 repetitions on 3 qubits produces:



(using CZ gates, shown as controlled-Z with control dots on both qubits)

the key insight: RealAmplitudes is just TwoLocal with rotation_blocks= $[R_y]$ and entanglement_blocks=[CNOT]. EfficientSU2 is TwoLocal with rotation_blocks= $[R_y, R_z]$ and entanglement_blocks=[CNOT].

10 Hardware-Efficient vs Problem-Inspired Ansatz

theres an important distinction between hardware-efficient ansatz (designed for the hardware) and problem-inspired ansatz (designed for the problem). lets compare.

10.1 The Tradeoff

Property	Hardware-Efficient	Problem-Inspired
design principle	minimize hardware cost	encode problem structure
examples	RealAmplitudes, EfficientSU2	UCCSD, QAOA, HVA
native gates?	yes (by construction)	usually no
respects topology?	yes (by construction)	usually no
circuit depth (before transpile)	low	moderate to high
circuit depth (after transpile)	low (minimal change)	high (SWAP overhead)
expressibility	generic, untargeted	targeted to problem
trainability	often barren plateaus	better gradients
physical motivation	none	symmetries, conservation laws
parameter efficiency	low (many params, generic)	high (fewer params, targeted)

Key Point. the big surprise from the literature (mcclean et al. 2018): randomly structured hardware-efficient ansatzes tend to have barren plateaus — the gradient vanishes exponentially with the number of qubits. problem-inspired ansatzes often avoid this bcs they start with a structure thats already close to the solution.

but on noisy hardware, the problem-inspired ansatz might be too deep to run, while the hardware-efficient one actually executes. so the practical choice depends on ur specific hardware noise budget.

10.2 When to Use Which

use hardware-efficient when:

- ur working on NISQ hardware with tight noise budgets
- u dont have strong prior knowledge about the problem structure
- ur problem is a generic classification or regression task
- u need fast iteration (short circuits = fast execution)

use problem-inspired when:

- the problem has known symmetries (e.g., particle number conservation in chemistry)
- the hardware can support the required depth (e.g., trapped ions, or error-mitigated superconducting)
- u need good trainability and dont want to fight barren plateaus
- accuracy matters more than speed

11 Reducing Two-Qubit Gate Count

since two-qubit gates r $\sim 10\times$ noisier than single-qubit gates, reducing their count is the most impactful optimization u can make.

11.1 Gate Cancellation

the simplest technique: adjacent inverse gates cancel.

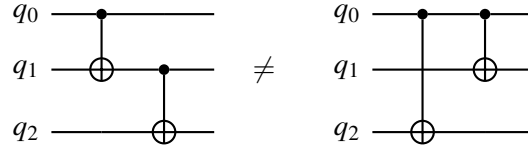
$$\text{CNOT}_{01} \cdot \text{CNOT}_{01} = I$$

this happens surprisingly often when circuits r composed from layers — the last CNOT of one layer and the first CNOT of the next might cancel.

11.2 Template Matching

certain circuit patterns can be replaced by shorter equivalents:

Example 11.1. the “CNOT cascade” identity:



(these r different circuits — template matching finds exact equivalences)

a real template: two CNOTs with a rotation between them can sometimes be simplified:

$$\text{CNOT}_{01} \cdot (I \otimes R_z(\theta)) \cdot \text{CNOT}_{01} = R_{zz}(\theta) \otimes I$$

(up to single-qubit corrections). this reduces 2 CNOTs to the equivalent of ~ 1 CNOT.

11.3 Commutation Analysis

gates that commute can be reordered to enable cancellations:

- R_z commutes with CNOT (on control qubit): $(\text{CNOT}_{01})(R_z(\theta) \otimes I) = (R_z(\theta) \otimes I)(\text{CNOT}_{01})$
- CZ is symmetric: $\text{CZ}_{01} = \text{CZ}_{10}$
- commuting gates through each other can bring adjacent gates together for cancellation

11.4 Two-Qubit Block Consolidation

Qiskit optimization level 3 does smtg clever: it identifies blocks of consecutive two-qubit gates, computes the unitary of the entire block (a 4×4 matrix), and resynthesizes it using the minimum number of native two-qubit gates via KAK decomposition.

for example, a sequence of 4 CNOTs between qubits 0 and 1 might consolidate to just 1 or 2 CNOTs plus single-qubit gates, bcs the combined unitary might not require all 3 CNOTs that a general $\text{SU}(4)$ needs.

12 QAOA as a Hardware-Aware Ansatz

the quantum approximate optimization algorithm (QAOA) is interesting bcs it sits between hardware-efficient and problem-inspired. lets see why.

12.1 QAOA Structure

recall from earlier lectures that QAOA for a cost function C uses the ansatz:

$$|\gamma, \beta\rangle = \prod_{p=1}^P e^{-i\beta_p \hat{B}} e^{-i\gamma_p \hat{C}} |+\rangle^{\otimes n}$$

where $\hat{B} = \sum_i X_i$ (mixer) and $\hat{C}$ encodes the problem.

for a MaxCut problem on graph $G = (V, E)$:

$$\hat{C} = \sum_{(i,j) \in E} \frac{1}{2} (I - Z_i Z_j)$$

the cost unitary decomposes as:

$$e^{-i\gamma\hat{C}} = \prod_{(i,j) \in E} e^{-i\gamma \frac{1}{2}(I - Z_i Z_j)} = \prod_{(i,j) \in E} R_{ZZ}(\gamma)_{ij}$$

each $R_{ZZ}(\gamma)$ requires one CNOT (or native two-qubit gate) plus single-qubit rotations.

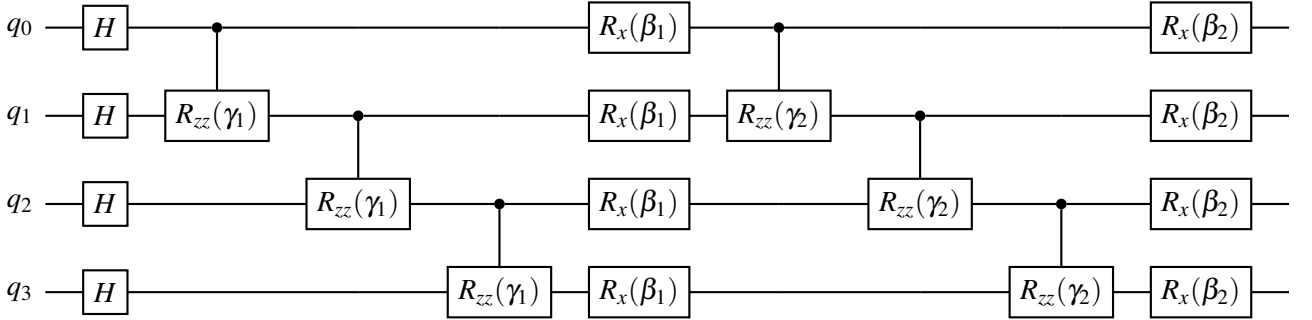
12.2 Why QAOA Can Be Hardware-Efficient

heres the key insight: if the problem graph G matches the hardware topology, then QAOA is automatically hardware-efficient.

Example 12.1. consider MaxCut on a graph thats isomorphic to the hardware connectivity graph (e.g., a heavy-hex graph on IBM hardware). then every R_{ZZ} gate in the cost unitary corresponds to a physically connected qubit pair — no SWAP routing needed. the circuit depth per QAOA layer is $O(1)$ (since gates on different edges can be parallelized).

Key Point. for optimization problems where the problem graph naturally matches the hardware topology, QAOA is both problem-inspired AND hardware-efficient. this is the best of both worlds. in general, QAOA’s hardware cost scales with the “distance” between the problem graph and the hardware graph.

the QAOA circuit for MaxCut on 4 qubits with linear connectivity (q_0 - q_1 - q_2 - q_3):



(here $R_{zz}(\gamma)$ between control and target is shorthand for the ZZ interaction implemented via CNOT- R_z -CNOT)

13 Practical Ansatz Design Checklist

when designing an ansatz for a specific hardware backend, follow this checklist:

13.1 Step 1: Know Ur Hardware

1. wt is the native gate set? (check backend configuration)
2. wt is the connectivity map? (check coupling map)
3. wt r the current gate errors? (check calibration data)
4. wt r the coherence times? (check T_1 , T_2 per qubit)
5. which qubits r best right now? (check latest calibration)

13.2 Step 2: Compute Ur Noise Budget

estimate the maximum circuit depth:

$$D_{\max} \approx \frac{-\ln(\epsilon_{\text{target}})}{n_{2Q} \cdot \epsilon_{2Q} + n_{1Q} \cdot \epsilon_{1Q}}$$

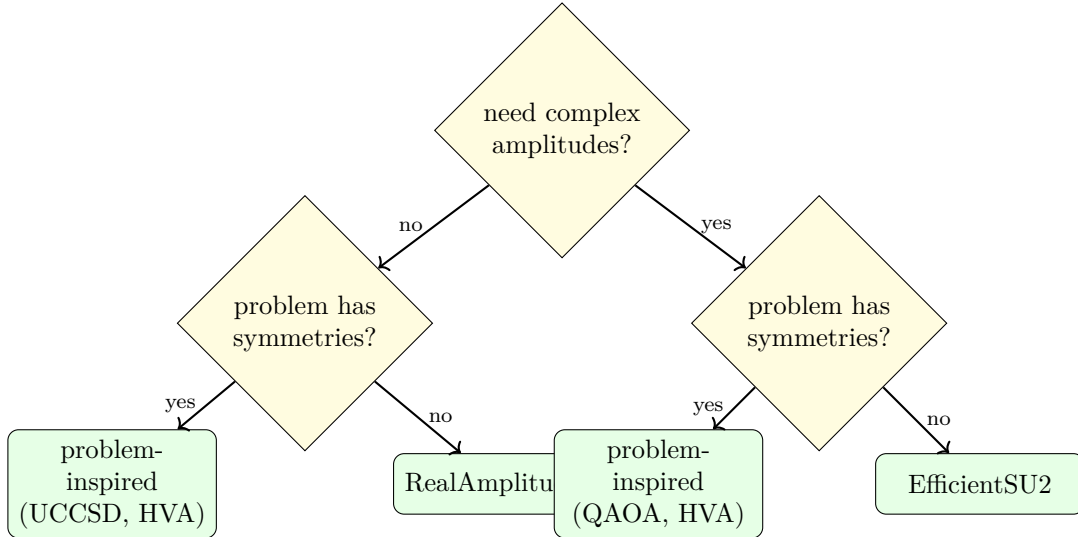
where ϵ_{target} is ur acceptable total error (e.g., 0.3 for a QML classifier).

Example 13.1. for IBM Heron with $\epsilon_{2Q} = 0.003$, $\epsilon_{1Q} = 0.0003$, 10 qubits:

$$D_{\max} \approx \frac{-\ln(0.3)}{9 \times 0.003 + 10 \times 0.0003} \approx \frac{1.2}{0.027 + 0.003} = \frac{1.2}{0.03} = 40 \text{ layers}$$

so u can afford about 40 entangling layers. thats actually quite generous for a hardware-efficient ansatz.

13.3 Step 3: Choose Ansatz Family



13.4 Step 4: Configure Entanglement Pattern

match the entanglement pattern to ur hardware:

- linear chain topology: use “linear” entanglement (neighbor pairs)
- heavy-hex topology: design custom entanglement map matching the hardware edges
- all-to-all (trapped ions): use “full” entanglement for maximum expressiveness
- square grid: use “pairwise” entanglement alternating horizontal and vertical edges

13.5 Step 5: Set Number of Layers

start small and increase:

1. start with $L = 1$ (minimal ansatz)

2. train and evaluate. if the model underfits, increase L
3. keep increasing until u hit one of: (a) good performance, (b) overfitting, (c) noise budget limit, (d) barren plateaus (gradients vanish)
4. typical good range: $L = 2-6$ for current NISQ hardware

13.6 Step 6: Transpile and Verify

1. transpile ur circuit at optimization level 3
2. check the native gate count (especially 2Q gates)
3. verify the native depth is within ur noise budget
4. if its too deep, reduce layers or simplify entanglement
5. compare with a random circuit of the same structure to check for barren plateaus

14 The Qiskit Pass Manager

for fine-grained control over transpilation, Qiskit provides the pass manager framework. understanding it helps u optimize beyond what the default `transpile()` function does.

14.1 Transpilation Passes

a “pass” is a single transformation applied to a circuit. passes r grouped into stages:

1. init stage: high-level optimization before layout (e.g., unrolling custom gates)
2. layout stage: map logical qubits to physical qubits (e.g., TrivialLayout, SabreLayout, DenseLayout)
3. routing stage: insert SWAPs for non-local gates (e.g., SabreSwap, StochasticSwap)
4. translation stage: convert to native gates (e.g., BasisTranslator, UnrollCustomDefinitions)
5. optimization stage: reduce gate count (e.g., Optimize1qGates, CXCancellation, CommutativeCancellation)
6. scheduling stage: add timing information for pulse-level execution

14.2 Optimization Passes in Detail

the key optimization passes for reducing 2Q gate count:

CXCancellation: finds and removes adjacent CNOT pairs:

$$\text{CNOT}_{01} \cdot \text{CNOT}_{01} \rightarrow I$$

CommutativeCancellation: identifies gates that commute, reorders them, and cancels:

$$\text{CNOT}_{01} \cdot (R_z \otimes I) \cdot \text{CNOT}_{01} \rightarrow R_{zz} \text{ equivalent}$$

Collect2qBlocks + ConsolidateBlocks: groups consecutive 2Q gates into blocks, computes the unitary, and resynthesizes with minimum gates.

TemplateOptimization: matches known circuit identities and applies simplifications.

Example 14.1 (Custom Pass Manager for QML). a practical pass manager for QML circuits:

1. use SabreLayout with multiple random seeds (try 20–50 layouts, keep the best)
2. use SabreSwap for routing
3. run Collect2qBlocks + ConsolidateBlocks to minimize 2Q gates
4. run CommutativeCancellation
5. run Optimize1qGatesDecomposition to merge single-qubit gates
6. repeat optimization passes 2–3 times (each pass can enable new optimizations)

15 Circuit Depth Optimization Techniques

beyond the transpiler, there r several techniques u can use to reduce circuit depth at the design level.

15.1 Parallelization

gates on non-overlapping qubits can execute simultaneously. design ur entanglement layers to maximize parallelism:

bad (sequential):

$$\text{CNOT}_{01} \rightarrow \text{CNOT}_{12} \rightarrow \text{CNOT}_{23} \rightarrow \text{CNOT}_{34} \quad (\text{depth } 4)$$

good (parallel):

$$\{\text{CNOT}_{01}, \text{CNOT}_{23}\} \rightarrow \{\text{CNOT}_{12}, \text{CNOT}_{34}\} \quad (\text{depth } 2)$$

for “linear” entanglement in Qiskit, the gates r automatically layered for parallelism using even/odd pairs.

15.2 Entanglement Reduction

instead of full (all-to-all) entanglement, use sparser patterns:

Pattern	2Q Gates/Layer	Depth/Layer	Expressibility
linear	$n - 1$	2	moderate
circular	n	2–3	good
pairwise	$\lfloor n/2 \rfloor$	1	low per layer
full	$\binom{n}{2}$	$n - 1$	highest

Key Point. for QML on NISQ hardware, “linear” entanglement with more layers is usually better than “full” entanglement with fewer layers. u get similar expressiveness with much less noise bcs every gate is between adjacent qubits.

15.3 Layer-Wise Training

instead of training all layers simultaneously (which can lead to barren plateaus), train one layer at a time:

1. initialize a 1-layer ansatz, train to convergence
2. add a second layer (initialized to identity or small random values), train
3. repeat until adding more layers doesnt improve performance

this doesnt reduce the final circuit depth, but it makes training much more reliable and often finds better optima.

15.4 Parameter Sharing

use the same parameter values across different layers:

$$\theta_i^{(l)} = \theta_i \quad \forall l$$

this reduces the parameter count from $n(L+1)$ to n while keeping the same circuit depth. the expressiveness is reduced, but this can actually help with trainability (fewer parameters = simpler optimization landscape).

16 Impact of Transpilation: Detailed Example

lets work through a complete example of how transpilation transforms a QML circuit.

16.1 The Abstract Circuit

consider a 4-qubit EfficientSU2 ansatz with 2 layers and circular entanglement. the abstract circuit has:

- $3 \times 2 \times 4 = 24$ single-qubit rotation gates (R_y and R_z)
- $2 \times 4 = 8$ CNOT gates (circular: q_0-q_1 , q_1-q_2 , q_2-q_3 , q_3-q_0 , repeated twice)
- abstract depth: approximately 12

16.2 Transpilation to Linear Chain

on a linear chain ($q_0-q_1-q_2-q_3$), the q_3-q_0 CNOT in each layer requires SWAP routing. lets trace through:

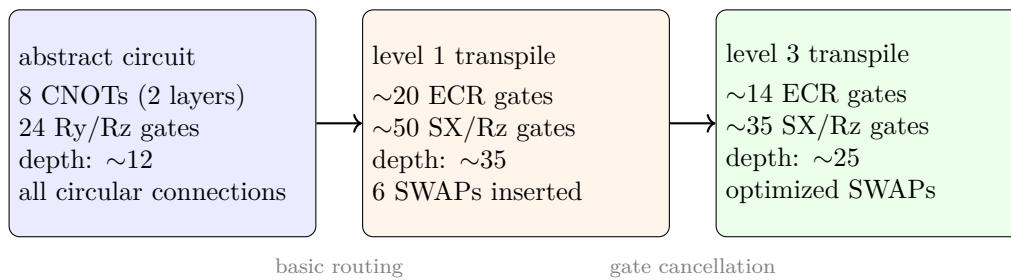
layer 1 entanglement:

- CNOT(q_0, q_1): native, no change (1 ECR)
- CNOT(q_1, q_2): native, no change (1 ECR)
- CNOT(q_2, q_3): native, no change (1 ECR)

- CNOT(q_3, q_0): NOT native. need to route q_0 state to be adjacent to q_3 :
 - SWAP(q_0, q_1): 3 ECR gates
 - SWAP(q_1, q_2): 3 ECR gates
 - CNOT(q_2, q_3): 1 ECR gate
 - (then the qubit mapping has changed — the transpiler tracks this)
- subtotal: $3 + 7 = 10$ ECR gates (vs. 4 ideal)

after optimization (level 3), some of these ECR gates may cancel with subsequent gates, reducing the total. typical final count: ~ 7 – 8 ECR gates per layer.

16.3 Before/After Comparison



Key Point. the lesson: a circuit that's designed to be hardware-efficient from the start (using only linear entanglement) would have needed only 6 ECR gates per layer (3 per layer, 2 layers) with zero SWAP overhead. that's less than half the cost of the circular version after transpilation. always design for the hardware from the beginning.

17 Advanced: Noise-Aware Ansatz Design

beyond just respecting topology, you can go further and design ansatzes that account for the specific noise properties of each qubit and gate.

17.1 Qubit Quality Variation

on a real backend, different qubits and different qubit pairs have different error rates. a noise-aware design strategy:

1. query the backend calibration data
2. rank qubit pairs by two-qubit gate error
3. select a connected subgraph of the n best qubits
4. design your entanglement pattern to use only the lowest-error qubit pairs
5. avoid the worst qubits entirely (even if they're physically present)

Example 17.1. on a 127-qubit Eagle processor, the best 10-qubit subgraph might have:

- average 2Q error: 0.5% (vs. 1.2% chip average)

- average T_1 : 250 μs (vs. 150 μs chip average)
- this effectively doubles ur noise budget and circuit depth limit

17.2 Error-Aware Entanglement

instead of uniform entanglement patterns, weight the entangling gates by the quality of the underlying hardware:

1. for qubit pairs with low error: use CNOT gates (full entanglement)
2. for qubit pairs with moderate error: use parameterized $R_{ZZ}(\theta)$ gates (can reduce entanglement strength)
3. for qubit pairs with high error: skip the entangling gate entirely

this creates an ansatz where the entanglement structure is adapted to the noise landscape of the specific processor.

17.3 Dynamical Decoupling

between gate operations, qubits idle and decohere. dynamical decoupling (DD) inserts sequences of identity-equivalent pulses during idle periods to protect against low-frequency noise:

$$\text{DD sequence: } X - \tau - X - \tau - X - \tau - X - \tau$$

this refocuses slow dephasing noise (similar to a spin echo). Qiskit supports automatic DD insertion during transpilation.

for QML circuits, DD is especially important when ur circuit has unbalanced depth across qubits — some qubits might idle for many gate durations while others r busy.

18 Summary and Design Principles

key takeaways from this lecture:

1. hardware-efficient means: native gates, native connectivity, minimal depth. design for the hardware from the start, dont rely on the transpiler to fix a bad circuit.
2. native gate sets matter: use R_z (virtual, zero cost) and minimize physical gates. on IBM, target SX + ECR as ur building blocks.
3. SWAP gates r expensive: 3 CNOTs each, $\sim 1.5\%$ error. minimize non-local entanglement.
4. RealAmplitudes is ur starting point: simple, $n(L+1)$ parameters, real amplitudes only. upgrade to EfficientSU2 ($2n(L+1)$ parameters) if u need more expressiveness.
5. transpilation at level 3: always use it for production. it can reduce 2Q gate count by 20–50%.
6. QAOA is naturally hardware-efficient when the problem graph matches the hardware topology.

7. the noise budget rules everything: compute $D_{\max}$ for ur hardware. if ur ansatz after transpilation exceeds $D_{\max}$, simplify it. no amount of error mitigation can save a circuit thats 10x too deep.
8. check calibration data daily: hardware quality drifts. the best qubits today might not be the best tomorrow.
9. start simple, add complexity: begin with $L = 1$ RealAmplitudes on the best qubits. only add layers, parameters, or more complex ansatze when u have evidence that the simpler version isnt enough.

Key Point. the meta-lesson: in the NISQ era, the best QML model isnt the most theoretically powerful one — its the one that extracts the most useful computation from a limited noise budget. hardware awareness is not an afterthought; its the first design decision.

next lecture well apply these principles to run actual QML experiments on real hardware, dealing with all the practical issues: job management, error mitigation, result interpretation, and debugging when things go wrong (which they will).